

Skill Session 19

Task 1

To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events

```
j_rakshitha@RAKSHITHA:~$ nano session19_task1_handlers.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra session19_task1_handlers.c -o session19_task1_handlers
j_rakshitha@RAKSHITHA:~$ ./session19_task1_handlers
Practical Skill Session 19 - Task 1
=====
Signal handlers registered successfully.
PID: 1366

Commands:
  status - show signal status
  signal - send SIGUSR1 to this process
  exit   - quit normally

Press Ctrl+C to test SIGINT.

signal-shell> signal
Sending SIGUSR1 to PID 1366...

[HANDLER] SIGUSR1 received.
SIGUSR1 count: 1

signal-shell> ^C
[HANDLER] SIGINT received.
SIGINT count: 1
status

Signal Handler Status
=====
Process PID : 1366
SIGINT count: 1
SIGUSR1 count: 1
SIGTERM      : NOT RECEIVED
=====

signal-shell>
[HANDLER] SIGTERM received.
Preparing safe termination...

Program exiting safely.
j_rakshitha@RAKSHITHA:~$
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
j_rakshitha@RAKSHITHA:~$ kill -TERM 1366
j_rakshitha@RAKSHITHA:~$ |
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session19_task1_handlers.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>

volatile sig_atomic_t sigint_count = 0;
volatile sig_atomic_t sigusr1_count = 0;
volatile sig_atomic_t sigterm_received = 0;

void handle_sigint(int sig)
{
    (void)sig;
    sigint_count++;

    printf("\n[HANDLER] SIGINT received.\n");
    printf("SIGINT count: %d\n", sigint_count);
    fflush(stdout);
}

void handle_sigusr1(int sig)
{
    (void)sig;
    sigusr1_count++;

    printf("\n[HANDLER] SIGUSR1 received.\n");
    printf("SIGUSR1 count: %d\n", sigusr1_count);
    fflush(stdout);
}

void handle_sigterm(int sig)
{
    (void)sig;
    sigterm_received = 1;

    printf("\n[HANDLER] SIGTERM received.\n");
    printf("Preparing safe termination...\n");
    fflush(stdout);
}

void install_handler(int signal_number, void (*handler)(int))
{
    struct sigaction sa;

    memset(&sa, 0, sizeof(sa));
    sigemptyset(&sa.sa_mask);

    sa.sa_handler = handler;
    sa.sa_flags = SA_RESTART;

    if (sigaction(signal_number, &sa, NULL) == -1) {
        [ Read 116 lines ]
    }
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

```
j_rakshitha@RAKSHITHA: ~ × + v
GNU nano 8.7.1 session19_task1_handlers.c
    }
    exit(EXIT_FAILURE);
}

void show_status(void)
{
    printf("\nSignal Handler Status\n");
    printf("=====\n");
    printf("Process PID : %d\n", getpid());
    printf("SIGINT count: %d\n", sigint_count);
    printf("SIGUSR1 count: %d\n", sigusr1_count);
    printf("SIGTERM      : %s\n",
           sigterm_received ? "RECEIVED" : "NOT RECEIVED");
    printf("=====\n");
}

int main(void)
{
    char input[100];

    install_handler(SIGINT, handle_sigint);
    install_handler(SIGUSR1, handle_sigusr1);
    install_handler(SIGTERM, handle_sigterm);

    printf("Practical Skill Session 19 - Task 1\n");
    printf("=====\n");
    printf("Signal handlers registered successfully.\n");
    printf("PID: %d\n", getpid());

    printf("\nCommands:\n");
    printf("  status - show signal status\n");
    printf("  signal - send SIGUSR1 to this process\n");
    printf("  exit   - quit normally\n");
    printf("\nPress Ctrl+C to test SIGINT.\n");

    while (!sigterm_received) {
        printf("\nsignal-shell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            continue;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "status") == 0) {
            show_status();
        }
        else if (strcmp(input, "signal") == 0) {
            printf("Sending SIGUSR1 to PID %d...\n", getpid());

            if (kill(getpid(), SIGUSR1) == -1)
                perror("kill");
        }
        else if (strcmp(input, "exit") == 0) {
            break;
        }
        else if (strlen(input) > 0) {
            printf("Unknown command.\n");
        }
    }

    printf("\nProgram exiting safely.\n");
    return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

```
        perror("kill");
    }
    else if (strcmp(input, "exit") == 0) {
        break;
    }
    else if (strlen(input) > 0) {
        printf("Unknown command.\n");
    }
}

printf("\nProgram exiting safely.\n");
return 0;
}
```

Task 2

To Copy SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling

```
j_rakshitha@RAKSHITHA:~$ nano session19_task2_sigint.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra session19_task2_sigint.c -o session19_task2_sigint
j_rakshitha@RAKSHITHA:~$ ./session19_task2_sigint
Practical Skill Session 19 - Task 2
=====
SIGINT forwarding shell started.
Shell PID: 1602

Commands:
  run <seconds> - start a foreground job
  status        - show job state
  exit          - quit shell

Press Ctrl+C while a job is running to forward SIGINT.

sigint-shell> run 10
[Shell] Foreground job started: PID 1603
[Job] PID 1603 started for 10 seconds.
[Job] PID 1603 working... 1/10
^C
[Shell] SIGINT received.
[Shell] Forwarding SIGINT to foreground job PID 1603...
[Shell] Foreground job PID 1603 terminated by signal 2.
[Shell] Foreground job state updated: COMPLETED/TERMINATED

sigint-shell> status
[Shell] No active foreground job.

sigint-shell>

sigint-shell> exit

Shell exiting safely.
j_rakshitha@RAKSHITHA:~$ |
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session19_task2_sigint.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <string.h>
#include <errno.h>

volatile sig_atomic_t foreground_pid = -1;
volatile sig_atomic_t child_finished = 0;

void handle_sigint(int sig)
{
    (void)sig;

    printf("\n[Shell] SIGINT received.\n");

    if (foreground_pid > 0) {
        printf("[Shell] Forwarding SIGINT to foreground job PID %d...\n",
            foreground_pid);

        if (kill(foreground_pid, SIGINT) == -1) {
            perror("kill");
        }
    } else {
        printf("[Shell] No foreground job. Shell remains active.\n");
    }

    fflush(stdout);
}

void handle_sigchld(int sig)
{
    (void)sig;
    child_finished = 1;
}

int main(void)
{
    struct sigaction sa_int;
    struct sigaction sa_chld;
    char input[100];

    memset(&sa_int, 0, sizeof(sa_int));
    sigemptyset(&sa_int.sa_mask);
    sa_int.sa_handler = handle_sigint;
    sa_int.sa_flags = SA_RESTART;

    if (sigaction(SIGINT, &sa_int, NULL) == -1) {
        perror("sigaction SIGINT");
        return 1;
    }

    [ Read 170 lines ]
    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session19_task2_sigint.c
}

memset(&sa_chld, 0, sizeof(sa_chld));
sigemptyset(&sa_chld.sa_mask);
sa_chld.sa_handler = handle_sigchld;
sa_chld.sa_flags = SA_RESTART;

if (sigaction(SIGCHLD, &sa_chld, NULL) == -1) {
    perror("sigaction SIGCHLD");
    return 1;
}

printf("Practical Skill Session 19 - Task 2\n");
printf("=====\n");
printf("SIGINT forwarding shell started.\n");
printf("Shell PID: %d\n", getpid());

printf("\nCommands:\n");
printf("  run <seconds> - start a foreground job\n");
printf("  status      - show job state\n");
printf("  exit        - quit shell\n");
printf("\nPress Ctrl+C while a job is running to forward SIGINT.\n");

while (1) {
    printf("\nsigint-shell> ");
    fflush(stdout);

    if (fgetc(input, sizeof(input), stdin) == NULL)
        break;

    if (strncmp(input, "run ", 4) == 0) {
        int seconds;

        if (sscanf(input + 4, "%d", &seconds) != 1 ||
            seconds <= 0) {
            printf("Usage: run <seconds>\n");
            continue;
        }

        pid_t pid = fork();

        if (pid < 0) {
            perror("fork");
            continue;
        }

        if (pid == 0) {
            signal(SIGINT, SIG_DFL);

            printf("[Job] PID %d started for %d seconds.\n",
                getpid(), seconds);
        }
    }
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
j_rakshitha@RAKSHITHA: ~ × + v
GNU nano 8.7.1 session19_task2_sigint.c

    fflush(stdout);

    for (int i = 1; i <= seconds; i++) {
        printf("[Job] PID %d working... %d/%d\n",
               getpid(), i, seconds);
        fflush(stdout);
        sleep(1);
    }

    printf("[Job] PID %d completed normally.\n", getpid());
    fflush(stdout);
    exit(0);
}

foreground_pid = pid;
child_finished = 0;

printf("[Shell] Foreground job started: PID %d\n", pid);

int status;
while (1) {
    pid_t result = waitpid(pid, &status, WNOHANG);

    if (result == pid) {
        break;
    }

    if (result == -1) {
        if (errno == EINTR)
            continue;

        perror("waitpid");
        break;
    }

    sleep(1);
}

if (WIFSIGNALED(status)) {
    printf("[Shell] Foreground job PID %d terminated by signal %d.\n",
           pid, WTERMSIG(status));
} else if (WIFEXITED(status)) {
    printf("[Shell] Foreground job PID %d exited with status %d.\n",
           pid, WEXITSTATUS(status));
}

foreground_pid = -1;
printf("[Shell] Foreground job state updated: COMPLETED/TERMINATED\n");
}
else if (strcmp(input, "status\n") == 0) {
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo
```

```
    if (foreground_pid > 0)
        printf("[Shell] Foreground job PID %d is active.\n",
               foreground_pid);
    else
        printf("[Shell] No active foreground job.\n");
}
else if (strcmp(input, "exit\n") == 0) {
    break;
}
else if (strlen(input) > 1) {
    printf("Unknown command.\n");
}
}

printf("\nShell exiting safely.\n");
return 0;
}
```

Skill Session 20

Task 1

To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow

```
j_rakshitha@RAKSHITHA: ~  
j_rakshitha@RAKSHITHA:~$ nano session20_task1_sigtstp.c  
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra session20_task1_sigtstp.c -o session20_task1_sigtstp  
j_rakshitha@RAKSHITHA:~$ ./session20_task1_sigtstp  
Practical Skill Session 20 - Task 1  
=====  
SIGTSTP suspension shell started.  
Shell PID: 1698  
  
Commands:  
  run <seconds> - start a foreground job  
  jobs          - show job table  
  bg <job_id>   - resume a stopped job  
  exit         - quit shell  
  
sigtstp-shell> run 20  
[Shell] Started foreground job [1] PID=1701 PGID=1701  
[Job] PID 1701 working... 1/20  
[Job] PID 1701 working... 2/20  
^Z  
[Shell] SIGTSTP received.  
[Shell] Suspending foreground job PID 1701...  
[Shell] Job PID 1701 is now STOPPED.  
  
sigtstp-shell> jobs  
  
Job Table  
=====  
[1] PID=1701 PGID=1701 STATE=STOPPED  
=====  
  
sigtstp-shell> bg 1  
[Shell] Resuming job [1] with SIGCONT...  
[Shell] Job [1] resumed.  
  
sigtstp-shell> [Job] PID 1701 working... 3/20  
[Job] PID 1701 working... 4/20  
[Job] PID 1701 working... 5/20  
[Job] PID 1701 working... 6/20  
[Job] PID 1701 working... 7/20  
[Job] PID 1701 working... 8/20  
[Job] PID 1701 working... 9/20  
[Job] PID 1701 working... 10/20  
[Job] PID 1701 working... 11/20  
[Job] PID 1701 working... 12/20  
[Job] PID 1701 working... 13/20  
[Job] PID 1701 working... 14/20  
[Job] PID 1701 working... 15/20  
[Job] PID 1701 working... 16/20  
[Job] PID 1701 working... 17/20  
[Job] PID 1701 working... 18/20  
[Job] PID 1701 working... 19/20  
[Job] PID 1701 working... 20/20  
[Job] PID 1701 completed.  
jobs  
  
Job Table  
=====  
[1] PID=1701 PGID=1701 STATE=COMPLETED  
=====  
  
sigtstp-shell> exit  
Shell exiting safely.
```

```
j_rakshitha@RAKSHITHA: ~  
GNU nano 8.7.1 session20_task1_sigstsp.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <signal.h>  
#include <sys/wait.h>  
#include <string.h>  
#include <errno.h>  
  
#define MAX_JOBS 10  
  
typedef enum {  
    RUNNING,  
    STOPPED,  
    COMPLETED  
} JobState;  
  
typedef struct {  
    int id;  
    pid_t pid;  
    pid_t pgid;  
    JobState state;  
} Job;  
  
Job jobs[MAX_JOBS];  
int job_count = 0;  
  
volatile sig_atomic_t foreground_pid = -1;  
  
const char *state_name(JobState state)  
{  
    switch (state) {  
        case RUNNING:  
            return "RUNNING";  
        case STOPPED:  
            return "STOPPED";  
        case COMPLETED:  
            return "COMPLETED";  
        default:  
            return "UNKNOWN";  
    }  
}  
  
void handle_sigstsp(int sig)  
{  
    (void)sig;  
  
    printf("\n[Shell] SIGTSTP received.\n");  
  
    if (foreground_pid > 0) {  
        printf("[Shell] Suspending foreground job PID %d...\n",  
            foreground_pid);  
    }  
}
```

[Read 285 lines]

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

j_rakshitha@RAKSHITHA: ~

×

+

▼

—

□

×

GNU nano 8.7.1

session20_task1_sigtstp.c

```
        if (kill(foreground_pid, SIGSTOP) == -1) {
            perror("kill SIGSTOP");
        }
    } else {
        printf("[Shell] No foreground job to suspend.\n");
    }

    fflush(stdout);
}

void update_jobs(void)
{
    int status;

    for (int i = 0; i < job_count; i++) {
        if (jobs[i].state == COMPLETED)
            continue;

        pid_t result = waitpid(jobs[i].pid, &status,
                               WNOHANG | WUNTRACED);

        if (result == jobs[i].pid) {
            if (WIFSTOPPED(status)) {
                jobs[i].state = STOPPED;
            } else if (WIFEXITED(status) ||
                       WIFSIGNALED(status)) {
                jobs[i].state = COMPLETED;
            }
        }
    }
}

void show_jobs(void)
{
    update_jobs();

    printf("\nJob Table\n");
    printf("=====\n");

    if (job_count == 0) {
        printf("No jobs.\n");
    } else {
        for (int i = 0; i < job_count; i++) {
            printf("[%d] PID=%d PGID=%d STATE=%s\n",
                   jobs[i].id,
                   jobs[i].pid,
                   jobs[i].pgid,
                   state_name(jobs[i].state));
        }
    }
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

^C Location
^_ Go To Line

M-U Undo
M-E Redo

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session20_task1_sigsttp.c
printf("=====\n");
}

void run_job(int seconds)
{
    if (job_count >= MAX_JOBS) {
        printf("Job table is full.\n");
        return;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return;
    }

    if (pid == 0) {
        signal(SIGTSTP, SIG_DFL);
        setpgid(0, 0);

        for (int i = 1; i <= seconds; i++) {
            printf("[Job] PID %d working... %d/%d\n",
                getpid(), i, seconds);
            fflush(stdout);
            sleep(1);
        }

        printf("[Job] PID %d completed.\n", getpid());
        fflush(stdout);

        exit(0);
    }

    setpgid(pid, pid);

    jobs[job_count].id = job_count + 1;
    jobs[job_count].pid = pid;
    jobs[job_count].pgid = pid;
    jobs[job_count].state = RUNNING;

    foreground_pid = pid;

    printf("[Shell] Started foreground job [%d] PID=%d PGID=%d\n",
        jobs[job_count].id,
        pid,
        pid);

    job_count++;

    int status;

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session20_task1_sigtstp.c
while (1) {
    pid_t result = waitpid(pid, &status,
                           WUNTRACED);

    if (result == pid) {
        if (WIFSTOPPED(status)) {
            for (int i = 0; i < job_count; i++) {
                if (jobs[i].pid == pid) {
                    jobs[i].state = STOPPED;
                    break;
                }
            }

            printf("[Shell] Job PID %d is now STOPPED.\n", pid);
            break;
        }

        if (WIFEXITED(status) || WIFSIGNALED(status)) {
            for (int i = 0; i < job_count; i++) {
                if (jobs[i].pid == pid) {
                    jobs[i].state = COMPLETED;
                    break;
                }
            }

            printf("[Shell] Job PID %d completed.\n", pid);
            break;
        }
    }

    if (result == -1) {
        if (errno == EINTR)
            continue;

        perror("waitpid");
        break;
    }
}

foreground_pid = -1;
}

void resume_job(int id)
{
    update_jobs();

    for (int i = 0; i < job_count; i++) {
        if (jobs[i].id == id) {
            if (jobs[i].state != STOPPED) {
                printf("[Shell] Job [%d] is not stopped.\n", id);
                return;
            }
        }
    }
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session20_task1_sigtstp.c
}

printf("[Shell] Resuming job [%d] with SIGCONT...\n", id);

if (kill(jobs[i].pid, SIGCONT) == -1) {
    perror("kill SIGCONT");
    return;
}

jobs[i].state = RUNNING;

printf("[Shell] Job [%d] resumed.\n", id);
return;
}
}

printf("[Shell] Job [%d] not found.\n", id);
}

int main(void)
{
    char input[100];
    int seconds;
    int job_id;

    struct sigaction sa;

    memset(&sa, 0, sizeof(sa));
    sigemptyset(&sa.sa_mask);
    sa.sa_handler = handle_sigtstp;
    sa.sa_flags = SA_RESTART;

    if (sigaction(SIGTSTP, &sa, NULL) == -1) {
        perror("sigaction SIGTSTP");
        return 1;
    }

    printf("Practical Skill Session 20 - Task 1\n");
    printf("=====\n");
    printf("SIGTSTP suspension shell started.\n");
    printf("Shell PID: %d\n", getpid());

    printf("\nCommands:\n");
    printf("  run <seconds> - start a foreground job\n");
    printf("  jobs          - show job table\n");
    printf("  bg <job_id>   - resume a stopped job\n");
    printf("  exit          - quit shell\n");

    while (1) {
        printf("\nsigtstp-shell> ");
        fflush(stdout);

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
    if (fgets(input, sizeof(input), stdin) == NULL)
        break;

    if (sscanf(input, "run %d", &seconds) == 1) {
        if (seconds <= 0) {
            printf("Seconds must be greater than 0.\n");
        } else {
            run_job(seconds);
        }
    }
    else if (strcmp(input, "jobs\n") == 0) {
        show_jobs();
    }
    else if (sscanf(input, "bg %d", &job_id) == 1) {
        resume_job(job_id);
    }
    else if (strcmp(input, "exit\n") == 0) {
        break;
    }
    else if (strlen(input) > 1) {
        printf("Unknown command.\n");
    }
}

printf("\nShell exiting safely.\n");
return 0;
}
```

^G Help

^X Exit

^O Write Out

^R Read File

^F Where Is

^_ Replace

^K Cut

^U Paste

^T Exec

^J Just

Task 2

To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior

```
j_rakshitha@RAKSHITHA: ~ × + ∨
j_rakshitha@RAKSHITHA:~$ nano session20_task2_process_groups.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra session20_task2_process_groups.c -o session20_task2_process_groups
j_rakshitha@RAKSHITHA:~$ ./session20_task2_process_groups
Practical Skill Session 20 - Task 2
=====
Process group and terminal control shell started.
Shell PID: 1770
Shell PGID: 1770

Commands:
  run <seconds> - create a process group and transfer terminal control
  exit          - quit shell

pg-shell> run 5
[Shell] Created job PID=1771 PGID=1771
[Shell] Shell PGID=1770
[Shell] Transferring terminal control to PGID=1771...
[Job] PID=1771 PGID=1771 started.
[Job] PID=1771 working... 1/5
[Job] PID=1771 working... 2/5
[Job] PID=1771 working... 3/5
[Job] PID=1771 working... 4/5
[Job] PID=1771 working... 5/5
[Job] PID=1771 completed.
[Shell] Restoring terminal control to PGID=1770...

[1]+  Stopped                  ./session20_task2_process_groups
j_rakshitha@RAKSHITHA:~$ fg
./session20_task2_process_groups
[Shell] Job exited with status 0.

pg-shell> run 3
[Shell] Created job PID=1788 PGID=1788
[Shell] Shell PGID=1770
[Shell] Transferring terminal control to PGID=1788...
[Job] PID=1788 PGID=1788 started.
[Job] PID=1788 working... 1/3
[Job] PID=1788 working... 2/3
[Job] PID=1788 working... 3/3
[Job] PID=1788 completed.
[Shell] Restoring terminal control to PGID=1770...

[1]+  Stopped                  ./session20_task2_process_groups
j_rakshitha@RAKSHITHA:~$ fg
./session20_task2_process_groups
[Shell] Job exited with status 0.

pg-shell> exit

Shell exiting safely.
j_rakshitha@RAKSHITHA:~$ |
```

j_rakshitha@RAKSHITHA: ~

+

×

□

×

GNU nano 8.7.1session20_task2_process_groups.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <string.h>
#include <errno.h>
#include <termios.h>

void run_job(int seconds)
{
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return;
    }

    if (pid == 0) {
        /* Create a new process group */
        setpgid(0, 0);

        printf("[Job] PID=%d PGID=%d started.\n", getpid(), getpgrp());
        fflush(stdout);

        for (int i = 1; i <= seconds; i++) {
            printf("[Job] PID=%d working... %d/%d\n",
                getpid(), i, seconds);
            fflush(stdout);
            sleep(1);
        }

        printf("[Job] PID=%d completed.\n", getpid());
        fflush(stdout);

        exit(0);
    }

    /* Parent assigns the child's process group */
    setpgid(pid, pid);

    pid_t pgid = getpgid(pid);

    printf("[Shell] Created job PID=%d PGID=%d\n", pid, pgid);

    /* Save the shell's process group */
    pid_t shell_pgid = getpgrp();

    printf("[Shell] Shell PGID=%d\n", shell_pgid);

    /* Transfer terminal control to the job */

```

[Read 124 lines]

^G Help

^X Exit

^O Write Out

^R Read File

^F Where Is

^_ Replace

^K Cut

^U Paste

^T Execute

^J Justify

^C Location

^_ Go To Line

M-U Undo

M-E Redo

```
j_rakshitha@RAKSHITHA: ~ × + v
GNU nano 8.7.1 session20_task2_process_groups.c
printf("[Shell] Transferring terminal control to PGID=%d...\n", pgid);

if (tcsetpgrp(STDIN_FILENO, pgid) == -1) {
    perror("tcsetpgrp");
}

int status;

if (waitpid(pid, &status, WUNTRACED) == -1) {
    if (errno != EINTR)
        perror("waitpid");
}

/* Restore terminal control to the shell */
printf("[Shell] Restoring terminal control to PGID=%d...\n",
       shell_pgid);

if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1) {
    perror("tcsetpgrp");
}

if (WIFEXITED(status)) {
    printf("[Shell] Job exited with status %d.\n",
           WEXITSTATUS(status));
}
else if (WIFSIGNALED(status)) {
    printf("[Shell] Job terminated by signal %d.\n",
           WTERMSIG(status));
}
}

int main(void)
{
    char input[100];
    int seconds;

    printf("Practical Skill Session 20 - Task 2\n");
    printf("=====\n");
    printf("Process group and terminal control shell started.\n");
    printf("Shell PID: %d\n", getpid());
    printf("Shell PGID: %d\n", getpgrp());

    printf("\nCommands:\n");
    printf("  run <seconds> - create a process group and transfer terminal control\n");
    printf("  exit          - quit shell\n");

    while (1) {
        printf("\npg-shell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            continue;

        ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
        ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
    
```

```
        break;

    if (sscanf(input, "run %d", &seconds) == 1) {
        if (seconds <= 0) {
            printf("Seconds must be greater than 0.\n");
        }
        else {
            run_job(seconds);
        }
    }
    else if (strcmp(input, "exit\n") == 0) {
        break;
    }
    else if (strlen(input) > 1) {
        printf("Unknown command.\n");
    }
}

printf("\nShell exiting safely.\n");

return 0;
}
```